

哈爾濱工業大學（深圳）

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

操作系统 (Operating System)

操作系统实验

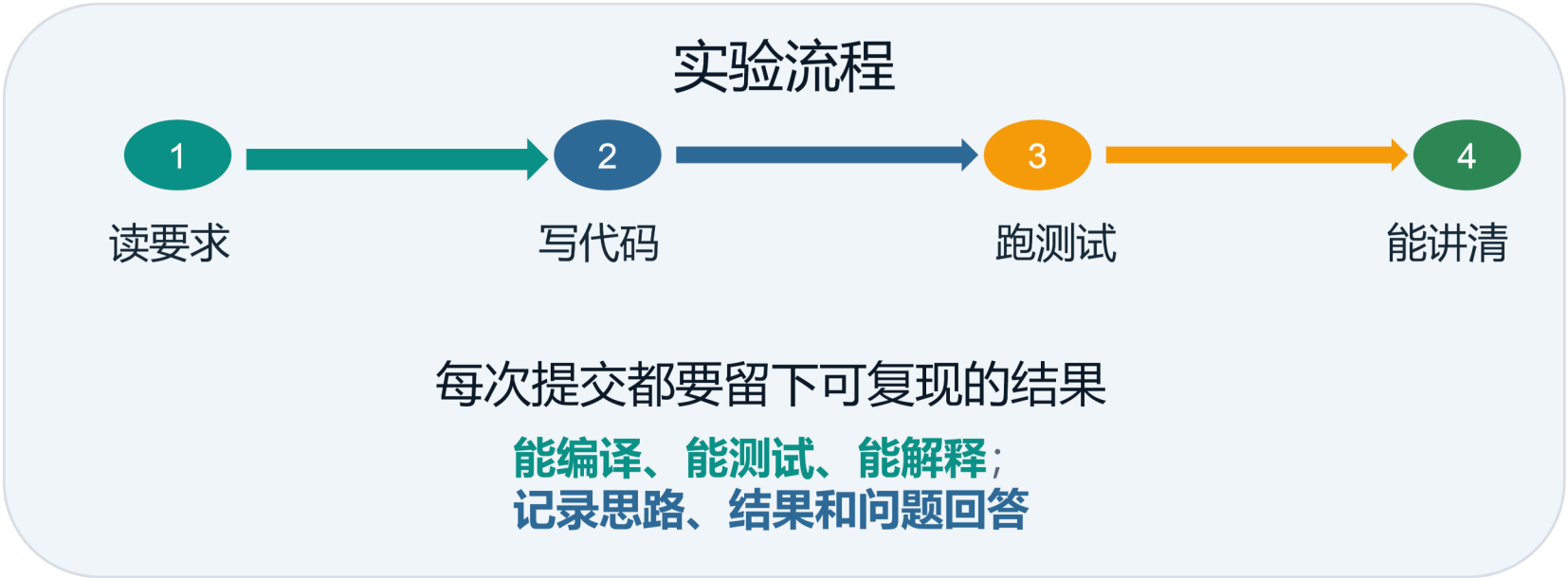
2026年秋季

目录



本学期实验总体安排

- 总成绩为30分，含现场考核与实验报告及代码。
- 实验项目：5个实验项目。
- 排课有冲突的同学可选择去其他班级补上，作业提交在原班级。



自己想过、自己做过、自己改过、自己讲得清

本学期实验总体安排

■ 每次 **4** 学时，共 **24** 学时；第 **4**、**6** 次课包含现场验收

课次	实验内容	学时	课堂重点
1	Lab1：XV6 与 Unix 实用程序	4	用户程序、启动流程、命令行 GDB
2	Lab2：进程与系统调用	4	系统调用与进程生命周期
3	Lab3：锁机制的应用	4	内存分配器与磁盘缓存优化
4	Lab4：页表	4	页表机制；**现场验收 Lab1～Lab3**
5	Lab5：基于 FUSE 的 EXT2 文件系统	4	文件系统设计与实现
6	**现场验收 Lab4～Lab5**	4	代码讲解、问答与必要的现场修改

主线：从用户态程序，逐步走到文件系统
先理解接口，再追踪实现，逐步处理进程并发、虚拟内存和持久化带来的复杂性

本学期实验课程考核

- **现场考核（30%）：代码讲解、问答，必要时现场修改**
  - **第一次考核**（第4次课）：Lab1~3实验完成情况，约占总成绩15%。
  - **第二次考核**（第6次课）：Lab4~5实验完成情况，约占总成绩15%。
- **代码及报告（70%）：**
  - 所有实验都要提交实验报告：**记录设计思路、运行结果与问题回答**
  - 代码提交要求：**要求commit.patch能够在初始代码上还原实验结果**
    - ✓Lab1~4仅需提交commit.patch差异文件；**补丁无法还原实验结果时，会影响评测与评分**
    - ✓Lab5需打包提交你实际修改过的源码和配置文件。

本学期实验课程考核 | 附加题

- **完成 5 个基础实验后，感兴趣且时间充裕的同学，可以继续挑战 5 个附加题：**
  - 完成全部附加题：**加实验分 30 分 + 平时分 10 分**；
  - 完成其中任意 1 个及以上：酌情加分

HITSZ osLabs：3个

Lab2附加题	Lab3附加题	Lab5附加题
AI Agent 沙箱 三个任务	AI 推理服务 四个任务	CheckpointFS、 VectorFS、 训练简单模型

MIT OS 6.S081/2020版本：2个

Copy-on-Write	mmap
理解页的共享、 复制与写时分离	理解虚拟地址到文 件映射的关系

- **附加题要求：**

① 提交能通过实验自测的完整**项目代码**；

② 提交完整的**项目分析报告，**包含对附加题的**内容分析、设计方法、算法分析**等，格式不限；

③ 由老师审核项目代码和分析报告，并参加**现场验收**确认附加题完成情况。

课程平台

- **课程指导书：** https://os-labs.p.cs-lab.top/
- **实验提交：** https://grader.hitsz.edu.cn/
- **实验代码：** https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
- **实验课件、报告及工具：** https://file.cs-lab.top/s/xcAKpIRmnrFUOPe
- **MIT 2020 XV6：** https://pdos.csail.mit.edu/6.828/2020/index.html

实验指导书首页

2. 实验相关链接

- 实验代码下载地址：https://gitee.com/ftutorials/xv6-oslabs-hitsz.git
- 实验课件、报告及工具下载地址（需校内网）：https://file.cs-lab.top/s/xcAKpIRmnrFUOPe
- 实验提交地址（需校内网）：https://grader.hitsz.edu.cn
- 2020 版本的 XV6 实验仓库下载地址：https://gitee.com/ftutorials/xv6-labs-2020.git

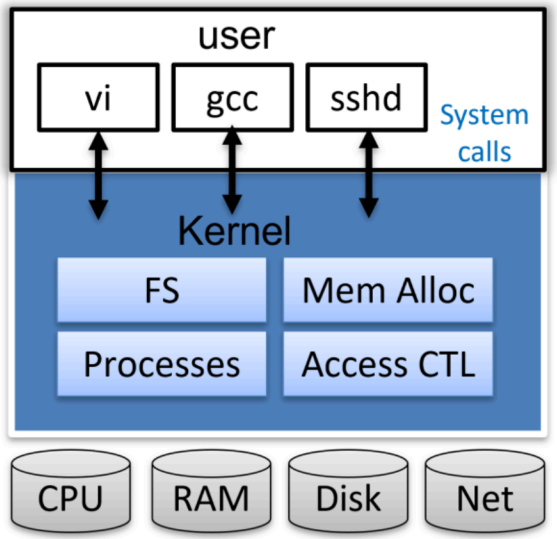
实验环境

- **实验室预装虚拟机（ Ubuntu + XV6 工具链已装好，即开即用）**
  - T2102/ T2210/T2507/T2608已预装
- **个人电脑虚拟机（本地持久化，离线也能练习）**
  - 需要导入 OVA 并预留磁盘空间
- **远程实验平台（已预装 XV6 完整环境）**
  - 并发性能有限，大量同学同时使用可能出现卡顿、断连
- **自行部署环境 （完全控制/本地访问）**
  - 手动配置

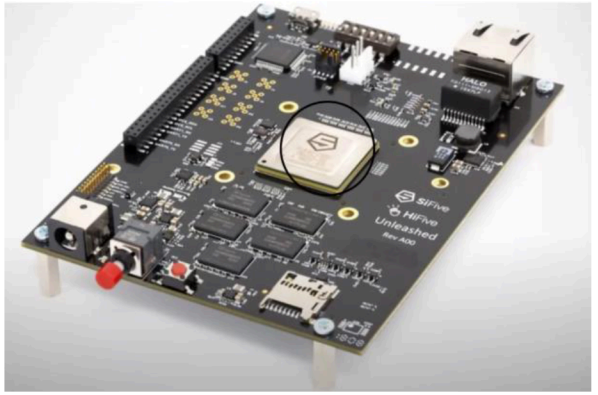
第一次课建议：先用实验室虚拟机或远程平台跑通 make qemu，再决定是否迁移到个人环境

实验环境

- **XV6**：基于**RISC-V**架构的操作系统
- **RISC-V工具链**：**riscv-gnu-toolchain**
  - 包含编译器（如GCC、LLVM）、汇编器、链接器、调试器（如GDB）等
- **QEMU**：开源的虚拟机监控器和仿真器
  - 允许用户在一台物理机上模拟出多台虚拟计算机。



比如用QEMU模拟一块RISCV开发板，这样就能在电脑上运行XV6了。



实验辅助资源

XV6讲解视频链接

- 我校OS课题组、各级学长助教们合力为大家录制的xv6讲解视频
  - ✓【HITSZ操作系统课程组讲解XV6（一）启动过程】
  - ✓【HITSZ操作系统课程组讲解XV6（二）进程管理】
  - ✓【HITSZ操作系统课程组讲解XV6（三）内存管理】

- 观看B站上的MIT 6.S081/Fall 2020课程视频
  - ✓通过学习这门课程可以让你对xv6操作系统有一个全面的认识。

实验辅助资源

XV6调试视频链接

- GDB命令行调试：
  - ✓【使用gdb调试xv6启动流程（1）】
  - ✓【使用gdb调试xv6启动流程（2）】
- VSCode图形化界面调试：
  - ✓【1. VSCode调试xv6内核代码】
  - ✓【2. VSCode调试xv6用户代码】
  - ✓【3. VSCode调试系统调用过程（包含pagetable和汇编）】
  - ✓【4. VSCode调试系统调用——从内核到用户，再从用户返回内核】

Lab1 | 实验目标及任务

- **熟悉** XV6 代码组织与开发流程
- **掌握**用户程序的编写、编译、运行和测试
- **理解**系统调用、进程间通信和目录遍历
- **观察** initcode 到 init 的启动切换

四个部分：围绕“写用户程序 + 看 XV6 启动”展开

A	sleep	读参数、调用 sleep、回到 shell
B	pingpong	两条管道，父子进程双向通信
C	find	目录项读取、类型判断、递归查找
D	启动流程调试	命令行 GDB，打印 initcode 与 init

最终提交：实验报告 + commit.patch；　commands.gdb 放入仓库并纳入补丁

实验步骤 | 准备工作①

- **推荐顺序：** **搭建并验证实验环境 → 阅读Lab1任务、测试与提交要求 → 完成sleep → 完成pingpong → 完成find → 使用GDB调试XV6启动流程 → 统一测试与提交**

- **分清楚三种命令行环境：**

Linux 终端

登录实验登录即可进入
获取、构建、测试、提交

git clone …
git checkout util
make qemu
./grade-lab-util sleep
make grade

XV6 命令行

在Linux终端运行make qemu进入
运行你写的用户程序

sleep 10
pingpong
find . b
Ctrl+a，再按 x 退出 QEMU

GDB 命令行

在Linux终端运行make gdb进入
观察启动过程中的进程切换

b main
c / si / s / n
p cpus[$tp]->proc->name
source commands.gdb
退出 GDB：q

```
test_19@comp1:~/xv6-oslab23-hitsz$ ./grade-lab-util sleep
make: 'kernel/kernel' is up to date.
== Test sleep, no arguments == sleep, no arguments: OK (1.4s)
== Test sleep, returns == sleep, returns: OK (0.9s)
== Test sleep, makes syscall == sleep, makes syscall: OK (0.9s)
test_19@comp1:~/xv6-oslab23-hitsz$
```

```
init: starting sh
$ echo > b
$ find . b
./b
$
```

```
../xv6-oslab24-hitsz  util !?  17:11
make qemu-gdb CPUS=1
*** Now run 'gdb' in another window.
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0 -S -gdb tcp::26000
```

实验步骤 | 准备工作②

■ 在实验开始之前，强烈建议你先完成以下工作：

- 熟悉常见命令的使用，如 **echo**、**xargs**、**find**。
- 了解目录的使用。了解 **.**、**..**、**/**分别表示什么，熟悉常见的目录操作命令，如**mkdir**、**cd**。
- 了解重定向的使用，重定向即命令中的**<**和**>**，用于修改右侧命令的标准输入/输出。例如 **echo Hello world > file_a** 会将字符串Hello world输出至文件file_a，而不是打印在终端。
- 了解管道的使用。管道即命令中的 **|**，用于将左侧命令的标准输出传递给右侧命令的标准输入。
- 了解常见系统调用的使用。如 **fork**、**exit**、**wait**、**open**、**close**、**read/write**、**pipe**、**dup**。

实验步骤 | 部署环境

- **准备环境：**请参考实验指导书部署实验环境。
- **Clone最新代码到本地：** *git clone https://gitee.com/ftutorials/xv6-oslabs-hitsz.git*
- **由于仓库会不定时更新，进行实验前，请务必执行 git pull 以拉取最新代码。**
- **每个实验项目均在专属的分支上完成，实验一需要切换至util分支进行开发。**

```
$ cd xv6-oslabs-hitsz
$ git pull
$ git checkout util
```



实验步骤 | 部署环境

- 在代码总目录下输入make qemu
- 当可以看到“init: starting sh”的字样表示xv6已经正常启动，此时在“$”提示符后可输入xv6支持的shell命令。
- 请记住**QEMU**的退出方法
  - **先按“Ctrl+a”组合键**
  - **接着全部松开**
  - **再按下“x”键**

```
[cs@localhost xv6-labs-2020]$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp
3 -nographic -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-devi
ce,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2059
xargstest.sh   2 3 93
cat            2 4 24216
echo           2 5 23032
forktest       2 6 13264
grep           2 7 27520
init           2 8 23776
kill           2 9 22984
ln             2 10 22824
ls             2 11 26408
mkdir          2 12 23136
rm             2 13 23120
sh             2 14 41936
stressfs       2 15 23976
usertests      2 16 148408
grind          2 17 38088
wc             2 18 25304
zombie         2 19 22368
sleep          2 20 22944
pingpong       2 21 23896
primes         2 22 24536
find           2 23 26696
xargs          2 24 24744
console        3 25 0
$
1 sleep  init
2 sleep  sh

$
```

输入ls命令

输入Ctrl + p显示进程信息

实验步骤 | 编译并运行sleep

✓ **Step1.** 新建user/sleep.c 文件，如图编写代码和注释；

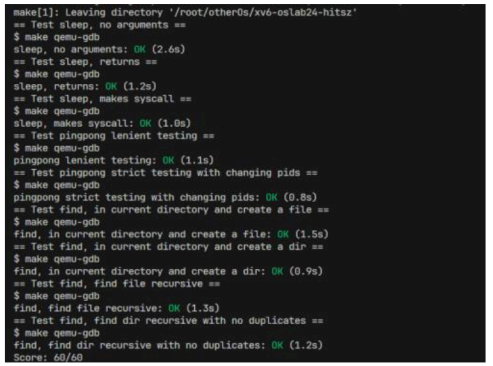
✓ **Step2.** 由于sleep.c 为新增的用户程序文件，请参考实验指导书修改Makefile文件；

✓ **Step3.** 编译xv6并运行sleep。

✓ **Step4.** 回答指导书中的相关问题。

```
xv6 kernel is booting

virtio disk init 0
hart 1 starting
hart 2 starting
init: starting sh
$ sleep 10
Sleep 10
$
```





先分清命令行：代码在编辑器里写，make 在 Linux 终端执行，sleep 在 XV6 命令行运行

实验步骤 | 编写用户程序，请参考实验提示完成

pingpong

1.父进程 -> 子进程：发送“ping”，子进程打印：
“<子PID>: received ping from pid <父PID>”

2.子进程 -> 父进程：发送“pong”，父进程打印：
“<父PID>: received pong from pid <子PID>”

```
init: starting sh
$ pingpong
4: received ping from pid 3
3: received pong from pid 4
$ pingpong
6: received ping from pid 5
5: received pong from pid 6
$ pingpong
8: received ping from pid 7
7: received pong from pid 8
$
```

find

1.在根目录新建**文件b**，在根目录下查询b的输出。

2.在根目录新建**目录b**，在根目录下查询b的输出。

3.**递归查询**：查询的目录或文件target在根目录的某个子文件夹

```
init: starting sh
$ mkdir a
$ echo > a/target
$ mkdir a/b
$ mkdir a/b/target
$ mkdir c
$ echo > c/target
$ find . target
./a/target
./a/b/target
./c/target
$
```

系统调用：fork • pipe • read • write • close • open • fstat • wait

实验步骤 | xv6启动流程实验

- **先抓住主线：硬件启动 → 内核初始化 → 第一个用户进程 → shell**
- **强烈建议观看以下视频：**
  - ✓ 理解xv6的启动流程：HITSZ操作系统课程组讲解XV6（一）
  - ✓ 学会使用 **gdb**调试：
    - ✓ 启动过程使用gdb调试xv6启动流程（1）
    - ✓ 使用gdb调试xv6启动流程（2）
- **GDB 检查点**：在关键位置执行 p cpus[$tp]->proc->name，分别观察 initcode 与 init
- **启动 QEMU**：make qemu-gdb CPUS=1；**另开终端**：make gdb；**汇编优先用 si，遇到 C 函数再用 s / n**
- 最终的 commands.gdb 文件（不限制长度，长度不算在分数内，但最短仅需6行，同学们可以尝试）应该类似这样：

```
...
p cpus[$tp]->proc->name # 应该先打印出 "initcode"
...
p cpus[$tp]->proc->name # 再打印出 "init"
da                      # 使用命令刷新 dashboard 并将上一个输出显示在 history 区域
```

```
History
$$1 = "initcode\000\000\000\000\000\000\000"
$$0 = "init\000\000de\000\000\000\000\000\000\000"
```

实验提交

■ 请务必关注grader作业平台，按时提交代码及报告

- ✓截止时间：两周内（**以作业平台截止时间为准，逾期补交将被扣分**）
- ✓提交要求：**commit.patch文件 + 实验报告**
- ✓提交前自检：

1	环境能启动	在正确的 XV6 代码仓库与 util 分支上，make qemu 能看到 $
2	Lab1 主线清楚	知道 sleep、pingpong、find 分别练什么，也知道 GDB 要观察什么
3	提交文件明确	报告 + commit.patch；commands.gdb 放在仓库并纳入补丁
4	结果可以复现	**补丁能在初始代码上应用**（参考如何检验commit.patch ）；测试、截图和结论来自自己的实验

现在开始：

进入实验环境，克隆仓库，

切换 util，先完成 sleep

Hope you enjoyed the OS course!